Assignment 8: Basic SQL Queries and Aggregate Functions

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: KUNGUMA KEERTHANA M

REGISTER NO: 2511021240051

COURSE: BCA

YEAR: 2ND YEAR

PROJECT TITLE: Online Food Delivery System

1. Aim

To retrieve, filter, sort, group, and summarize data from the Online Food Delivery System database using SQL queries and aggregate functions.

2. Software Requirements

- Database Management System: MySQL
- Tool: MySQL Workbench
- Language: SQL

3. Theory

SQL (Structured Query Language) is used to retrieve and manage data stored in relational databases.

- WHERE: Filters records based on a condition.
- ORDER BY: Sorts records in ascending or descending order.
- GROUP BY: Groups records with the same values.
- HAVING: Filters groups based on a condition.
- COUNT(): Counts the number of records.
- SUM(): Calculates the total of numeric values.
- AVG(): Calculates the average value.
- MIN(): Finds the smallest value.
- MAX(): Finds the largest value.

4. SQL Queries with Outputs and Explanations

The following queries use the sample data inserted in Assignment 7.

4.1 Filtering Records Using WHERE

SQL Query:

Input:

```
356 •   USE online_food_delivery;
357
358 •   SELECT Food_ID, Food_Name, Price
359     FROM Food_Item
360     WHERE Price > 150;
```

Output:

	Food_ID	Food_Name	Price
▶	10	Veg Biryani	180.00
	20	Paneer Butter Masala	220.00
	40	Margherita Pizza	250.00
	70	Veg Noodles	160.00
	80	Farmhouse Pizza	320.00
*	NULL	NULL	NULL

Explanation: The WHERE clause displays only food items whose prices are greater than ₹150.

4.2 Sorting Records Using ORDER BY

SQL Query:

Input:

```
362 •   SELECT Food_Name, Price
363     FROM Food_Item
364     ORDER BY Price DESC;
```

Output:

	Food_Name	Price
▶	Farmhouse Pizza	320.00
	Margherita Pizza	250.00
	Paneer Butter Masala	220.00
	Veg Biryani	180.00
	Veg Noodles	160.00
	South Indian Meals	150.00
	Veg Burger	120.00
	Chocolate Cake	100.00

Explanation: The ORDER BY clause sorts food items by price from highest to lowest. DESC means descending order.

4.3 Grouping Records Using GROUP BY

SQL Query:

Input:

```
366 •   SELECT Category_ID, COUNT(*) AS Total_Food_Items
367     FROM Food_Item
368     GROUP BY Category_ID
369     ORDER BY Category_ID;
```

Output:

Category_ID	Total_Food_Items
10	3
20	1
30	2
40	1
50	1

Explanation: The GROUP BY clause groups food items by category. COUNT(*) calculates the number of items in each category.

4.4 Filtering Groups Using HAVING

SQL Query:

Input:

```
371 •   SELECT Category_ID, COUNT(*) AS Total_Food_Items
372     FROM Food_Item
373     GROUP BY Category_ID
374     HAVING COUNT(*) > 1;
```

Output:

Category_ID	Total_Food_Items
10	3
30	2

Explanation: The HAVING clause displays only categories containing more than one food item. Unlike WHERE, it filters grouped results.

5. Aggregate Functions

5.1 COUNT() Function

SQL Query:

Input:

```
376 •   SELECT COUNT(*) AS Total_Customers
377     FROM Customer;
```

Output:

	Total_Customers
▶	6

Explanation: COUNT() returns the total number of customer records.

5.3 AVG() Function

SQL Query:

Input:

```
379 •   SELECT AVG(Price) AS Average_Food_Price
380     FROM Food Item;
```

Output:

	Average_Food_Price
▶	187.500000

Explanation: AVG() calculates the average price of all eight food items.

5.4 MIN() Function

SQL Query:

Input:

```
382 •   SELECT MIN(Price) AS Lowest_Food_Price
383     FROM Food_Item;
```

Output:

	Lowest_Food_Price
▶	100.00

Explanation: MIN() returns the lowest food price, which is ₹100.

5.5 MAX() Function

SQL Query:

Input:

```
385 •   SELECT MAX(Price) AS Highest_Food_Price
386     FROM Food_Item;
```

Output:

	Highest_Food_Price
▶	320.00

Explanation: MAX() returns the highest food price, which is ₹320.

6. Additional Query Using Multiple Aggregate Functions

SQL Query:

Input:

```
388 •   SELECT
389         COUNT(*) AS Total_Food_Items,
390         SUM(Price) AS Total_Food_Value,
391         AVG(Price) AS Average_Price,
392         MIN(Price) AS Minimum_Price,
393         MAX(Price) AS Maximum_Price
394     FROM Food_Item;
```

Output:

	Total_Food_Items	Total_Food_Value	Average_Price	Minimum_Price	Maximum_Price
▶	8	1500.00	187.500000	100.00	320.00

Explanation: This query demonstrates all five aggregate functions in a single SQL statement.

7. Procedure

1. Open MySQL Workbench and connect to the database server.
2. Select the Online_Food_Delivery database.
3. Ensure that the sample records from Assignment 7 have been inserted.
4. Execute each SQL query separately.
5. Compare the actual output with the expected output.
6. Capture screenshots of the SQL queries and their results for submission.

8. Result

SQL queries were written to filter, sort, and group database records. The HAVING clause was used to filter grouped results, and the aggregate functions COUNT(), SUM(), AVG(), MIN(), and MAX() were used to summarize the database information.

9. Conclusion

Basic SQL queries and aggregate functions help retrieve meaningful information from a database. These operations make it easier to analyze food prices, count records, summarize order amounts, and organize food items by category in the Online Food Delivery System.